

Project Executable Files

Date	02 October 2026
Team ID	SWTID-2026-1380
Project Name	PocketSmartAI - Your Smart Budget and recommendations assistant
Maximum Marks	3 Marks

PocketSmartAI – Project Executable Files

This section provides the executable and deployable materials for PocketSmartAI, a FastAPI-based smart planning and shopping web application. The application provides user authentication, AI-assisted planning, shopping recommendations, history, and budget-oriented catalog features. The source code is maintained in the Kaviya GitHub repository and the application is deployed as a web service on Render.

Step 1: Submission Checklist

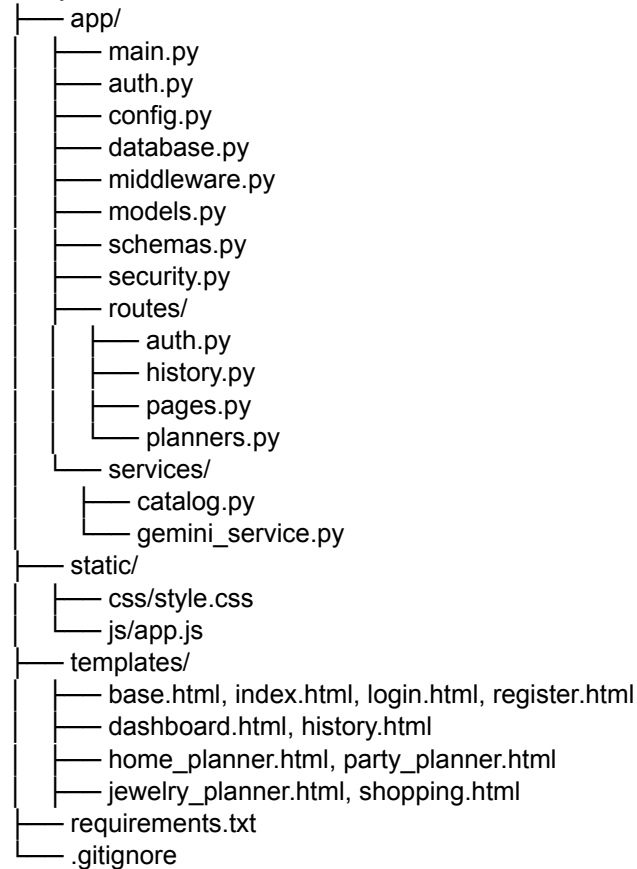
S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes / Included with project documentation
3	requirements.txt / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes – SQLite database/application database setup
5	Environment configuration file (.env.example or similar)	Required – secrets kept out of GitHub
6	Deployed application URL (if hosted)	Yes – Render
7	APK / Executable binary (if applicable)	NA – Web application
8	Dockerfile / Containerization config (if applicable)	NA

9	Test files and test results	Yes – JMeter performance evidence included in the Performance Testing document.
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Repository structure:

Kaviya/



Main technologies: Python, FastAPI, Uvicorn, Jinja2 templates, SQLite, Pydantic, Starlette sessions, and Gemini API integration.

/project-root (Kaviya)

/app - FastAPI application source code

/routes - Authentication, history, pages, and planner routes

/services - Catalog and Gemini AI services

main.py, auth.py, config.py, database.py, middleware.py

models.py, schemas.py, security.py

/static - CSS and JavaScript files

/css/style.css

/js/app.js

/templates - Jinja2 HTML templates

base.html, index.html, login.html, register.html
dashboard.html, history.html
home_planner.html, party_planner.html
jewelry_planner.html, shopping.html

requirements.txt - Python dependencies
.gitignore - Git configuration

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://pocketsmartai-kzfn.onrender.com/
Login Credentials (Demo)	Use the Register page to create a demo account. Do not store passwords in this document.
Platform / Hosting Provider	Render
Repository Link	https://github.com/9993082160-rgb/Kaviya
Demo Video Link	https://drive.google.com/drive/folders/1aMSNx-GM48YflrYe7gt9TdxTPCj9PJcX

Step 4: Run Instructions

1. Clone or download the Kaviya repository from GitHub.
2. Open a terminal in the repository root.
3. Create a Python virtual environment:
`python -m venv .venv`
4. Activate it on Windows PowerShell:
`.\.venv\Scripts\Activate.ps1`
5. Install dependencies:
`pip install -r requirements.txt`
6. Configure the required Gemini environment variables locally. Keep API keys private and do not commit them to GitHub.
7. Start the application:
`python -m uvicorn app.main:app --host 127.0.0.1 --port 8000`
8. Open `http://127.0.0.1:8000` in a browser.
9. Register/login and test the dashboard, planners, shopping, and history features.

For deployment, Render installs dependencies from `requirements.txt` and runs the FastAPI application using Uvicorn with the Render-provided PORT.

Pre-requisites:

- Python 3.10 or later
- Git
- Internet connection
- Gemini API key

Step 1: Clone the repository:

```
git clone https://github.com/9993082160-rgb/Kaviya.git
```

Step 2: Install dependencies:

```
pip install -r requirements.txt
```

Step 3: Configure environment:

Set `GEMINI_API_KEY` and `GEMINI_MODEL` as environment variables.

Step 4: Start the application:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Step 5: Open the application:

```
http://127.0.0.1:8000
```

Step 6: Register/Login and test the dashboard, planners, shopping, and history features.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Render Free web service may sleep after inactivity.	Open the site and allow time for the service to wake up.
2	Gemini features require a valid API key and configured model.	Configure GEMINI_API_KEY and GEMINI_MODEL as secure environment variables.
3	Local setup requires Python dependencies and environment configuration.	Follow the Run Instructions and install requirements.txt.

Performance Test Evidence

A JMeter Summary Report was captured for PocketSmartAI. The evidence records 20 samples, 558 ms average response time, 1003 ms maximum response time, 0.00% error rate, and 2.0 requests/sec throughput.